Step 3 — Scaling to 3 Tools With a Hard Step Limit and Failure Recovery — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: a one-tool demo never shows what breaks for real. Real tools call networks that fail, and real questions force the model to *choose* between tools. This step adds two more tools — one real API (weather) and one that fails on purpose — and the Worker from Step 2 handles them **without any change to its loop**: `run_tool()` already turns failures into observations, and the step limit already exists. Proving the loop absorbs real-world mess without edits is the whole payoff of having built it properly in Step 2. **If not:** the first real network failure would crash the app — or you'd be rewriting the loop now, instead of just adding tools.

After this step the Worker answers live weather, company policy and conversions, and recovers from a tool failure. Files not shown here don't change.

The new tools

Changed from Step 2: `exceptions.py` adds Doc02's two HTTP errors, used by the HTTP client below.

```
# src/exceptions.py
# The project's named errors: MissingConfigError (Doc01),
# TransientHTTPError and PermanentHTTPError (Doc02), and
# MaxIterationsExceeded (Doc07), which carries the partial step log.

class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):
    """Raised immediately for a non-retryable 4xx, with the response body
    attached."""
```

```
    def __init__(self, status_code, body):
        self.status_code = status_code
        self.body = body
        super().__init__(f"HTTP {status_code}: {body[:200]}")

class MaxIterationsExceeded(Exception):
    def __init__(self, message, steps):
        self.steps = steps
        super().__init__(message)
```

Story — `utils/http_client.py`: this is Doc02's Build Task HTTP client, with one change: the logger is fetched *inside* the function (like every other file here) so `LOG_LEVEL` from `.env` is loaded before logging starts. Reusing it means the weather tool gets timeouts and retries for free, instead of calling `requests` raw. **If not:** the weather tool would call `requests` directly, with no timeout and no retry, and one hung connection could freeze the agent.

```
# src/utils/http_client.py
# request_with_retry(): Doc02's HTTP client — a timeout on every call,
# retries with backoff for temporary errors, and a named error for each
# kind of failure.
import json
import random
import time

import requests

from src.exceptions import PermanentHTTPError, TransientHTTPError
from src.utils.logger import get_logger

def compute_backoff_delay(attempt):
    return (2 ** attempt) + random.uniform(0, 1)

def request_with_retry(method, url, max_attempts=5, **kwargs):
    logger = get_logger(__name__)
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=(3, 10), **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
```

```
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if response.status_code == 429 or response.status_code // 100 == 5:
            if attempt == max_attempts - 1:
                logger.error(f"gave up, last status {response.status_code}")
                raise TransientHTTPError(
                    f"status {response.status_code} "
                    f"after {max_attempts} attempts"
                )
            time.sleep(compute_backoff_delay(attempt))
            continue

        logger.error(f"permanent failure: status {response.status_code}")
        raise PermanentHTTPError(response.status_code, response.text)
```

Story — `tools/docs_search.py`: this is a small, local policy "search" with known answers — and it **fails on its first call on purpose**, using Doc07's `FlakyToolState`. That deliberate flakiness means every single run exercises the Worker's recovery path, instead of you hoping to catch it during a real outage. **If not:** failure recovery could only be tested by waiting for a real network outage to happen.

```
# src/tools/docs_search.py
# The policy search tool: search_docs(query) looks up the company's refund,
# shipping and warranty rules. It fails on its first call on purpose, so
# the Worker's recovery can be tested.
from langchain_core.tools import tool

COMPANY_DOCS = {
    "refund": "Refunds are accepted within 30 days of purchase, "
    "with the original receipt.",
    "shipping": "Standard shipping takes 3-5 working days; express "
    "shipping takes 1 working day.",
    "warranty": "Every product has a 1-year warranty against "
    "manufacturing faults.",
}

class FlakyToolState:
    def __init__(self):
```

```
        self.call_count = 0

search_state = FlakyToolState()

@tool
def search_docs(query):
    """Search the company's policy documents: refunds, shipping and
    warranty. Use it only for company policy questions."""
    search_state.call_count = search_state.call_count + 1
    if search_state.call_count == 1:
        raise ConnectionError("search index temporarily unavailable")

    lowered = query.lower()
    for topic in COMPANY_DOCS:
        if topic in lowered:
            return COMPANY_DOCS[topic]
    return "No matching document found."
```

Solution — a weather tool on Doc02's client, and a Worker that only gains tools

Story — `tools/weather.py` **:** this is Doc06's real-API tool, calling Doc02's `request_with_retry("GET", ...)` so it inherits timeouts and retries. Its description deliberately says when *not* to use it — because with three tools available, the tool **description** is literally how the model decides which one to call. **If not:** the model could reach for the weather tool on a policy question, and one slow response could hang the whole run.

```
# src/tools/weather.py
# The weather tool: get_weather(city) looks up the city's coordinates and
# calls the Open-Meteo API through request_with_retry().
from langchain_core.tools import tool

from src.utils.http_client import request_with_retry

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
    "islamabad": {"latitude": 33.68, "longitude": 73.05},
    "london": {"latitude": 51.51, "longitude": -0.13},
    "paris": {"latitude": 48.85, "longitude": 2.35},
}
```

```
def build_weather_params(city):
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return None
    return {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }

@tool
def get_weather(city):
    """Get the current temperature in Celsius for a city. Use it only for
    live weather questions — never for company policy or conversions."""
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."

    data = request_with_retry("GET", WEATHER_URL, params=params)
    temperature = data["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"
```

Changed from Step 2: `prompts/worker.txt` is rewritten for 3 tools and the retry rule.

Story — `prompts/worker.txt`: this spells out three things for the Worker: which kinds of questions actually need a tool, that a failed tool may be retried once, and that the final answer must come from what the tools returned (not the model's memory). With multiple tools now in play, these rules are what keep the Worker grounded and decisive. **If not:** the model might answer a policy question from memory, or give up after the very first error.

`src/prompts/worker.txt`

```
You are a research assistant. Use your tools for anything live, exact or
company-specific: weather, temperature conversions and company policy.
Answer general knowledge directly, without tools. If a tool returns an
error, you may try it again once. Base your final answer only on what the
tools returned, and say so if a tool could not give you the answer.
```

Changed from Step 2: in `worker.py`, **only** two imports and the `ALL_TOOLS` list change. The loop, `find_tool()`, `run_tool()`, the step log and the limit are exactly as you wrote them in Step 2.

```
# src/agents/worker.py
# The Worker agent: run_worker() is the ReAct loop — ask the model, run the
# tool it asks for, send the result back — with a step log and a hard
# step limit. ALL_TOOLS lists the tools it may use.
```

```
from dataclasses import dataclass, field

from src.exceptions import MaxIterationsExceeded
from src.tools.converter import celsius_to_fahrenheit
# new in Step 3: the 2 new tools
from src.tools.docs_search import search_docs
from src.tools.weather import get_weather
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

# changed in Step 3: 2 tools added to the list
ALL_TOOLS = [get_weather, celsius_to_fahrenheit, search_docs]

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

def find_tool(tools, name):
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

def run_tool(call, tools):
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        result = tool.invoke(call["args"])
        return str(result), None
    except Exception as e:
        return None, str(e)

def run_worker(model, task, max_iterations):
    logger = get_logger(__name__)
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [
        {"role": "system", "content": load_prompt("worker")},
        {"role": "user", "content": task},
    ]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
```

```
        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)

        for call in response.tool_calls:
            result, error = run_tool(call, ALL_TOOLS)
            if error is None:
                observation = result
                logger.info("step %s: %s -> %s", step, call["name"], result)
            else:
                observation = f"Error: {error}"
                logger.warning(
                    "step %s: %s failed -> %s", step, call["name"], error
                )
            messages.append({
                "role": "tool",
                "tool_call_id": call["id"],
                "content": observation,
            })
            steps.append({
                "step": step,
                "tool": call["name"],
                "arguments": call["args"],
                "result": result,
                "error": error,
            })

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

Changed from Step 2: `tests/unit/test_tools.py` adds cases 3, 4 and 5 — three new functions and their imports.

Story — `tests/unit/test_tools.py`: this adds cases that check the new tools with no network: cases 3 and 4 check the weather tool's coordinate lookup (a known city, and an unknown one), and case 5 proves the search tool fails once and then succeeds. Testing the tools in isolation keeps tool bugs from masquerading as agent bugs. **If not:** a broken tool would only show up indirectly, as a strange agent answer that's hard to trace.

```
# tests/unit/test_tools.py
# Checks the tools. Each case prints OK or WRONG.
# No API calls, no API key, no network needed.
# Run from the repo root: python -m tests.unit.test_tools
from pydantic import ValidationError

from src.tools.converter import celsius_to_fahrenheit
# new in Step 3
```

```
from src.tools.docs_search import search_docs, search_state
from src.tools.weather import build_weather_params

def test_converter():
    result = "OK"
    if celsius_to_fahrenheit.invoke({"celsius": 18}) != 64.4:
        result = "WRONG"
    print(f"case 1 (18°C is 64.4°F)           -> {result}")

def test_converter_rejects_bad_arguments():
    result = "WRONG"
    try:
        celsius_to_fahrenheit.invoke({"celsius": "hot"})
    except ValidationError:
        result = "OK"
    print(f"case 2 (celsius='hot' refused)    -> {result}")

# new in Step 3
def test_weather_params_known_city():
    params = build_weather_params("Lahore")
    result = "OK"
    if params is None or params["latitude"] != 31.55:
        result = "WRONG"
    print(f"case 3 (Lahore coordinates)       -> {result}")

# new in Step 3
def test_weather_params_unknown_city():
    result = "OK"
    if build_weather_params("Atlantis") is not None:
        result = "WRONG"
    print(f"case 4 (unknown city gives None)  -> {result}")

# new in Step 3
def test_search_fails_then_recovers():
    search_state.call_count = 0
    result = "WRONG"
    try:
        search_docs.invoke({"query": "refund policy"})
    except ConnectionError:
        result = "OK"
    second = search_docs.invoke({"query": "refund policy"})
    if "30 days" not in second:
        result = "WRONG"
    print(f"case 5 (fails once, then works)   -> {result}")

if __name__ == "__main__":
    test_converter()
```

```
    test_converter_rejects_bad_arguments()
    # new in Step 3
    test_weather_params_known_city()
    test_weather_params_unknown_city()
    test_search_fails_then_recovers()
```

Changed from Step 2: the integration test adds case 5 (a weather question uses `get_weather`) and case 6 (the Worker recovers from the failing search).

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API (and Open-Meteo) — each step
# adds cases. Each case prints OK or WRONG.
# Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents

from src.agents.worker import run_worker
from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.exceptions import MaxIterationsExceeded
from src.models.llm import get_model
# new in Step 3
from src.tools.docs_search import search_state

def test_direct_answer(model):
    chain = build_direct_chain(model)
    answer = chain.invoke({"question": "What is the capital of France?"})
    result = "OK"
    if "Paris" not in answer:
        result = "WRONG"
    print(f"case 1 (direct answer)  -> {result}: {answer}")

def tools_used(outcome):
    names = []
    for step in outcome.steps:
        names.append(step["tool"])
    return names

def test_conversion_uses_tool(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    used = tools_used(run_worker(model, task, 5))
    result = "OK"
    if "celsius_to_fahrenheit" not in used:
        result = "WRONG"
    print(f"case 2 (conversion)     -> {result}: {used}")

def test_no_tool_needed(model):
```

```
    outcome = run_worker(model, "What is the capital of France?", 5)
    result = "OK"
    if len(outcome.steps) != 0:
        result = "WRONG"
    print(f"case 3 (no tool needed) -> {result}: {outcome.final_answer}")

def test_step_limit(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    result = "WRONG"
    try:
        run_worker(model, task, 1)
    except MaxIterationsExceeded:
        result = "OK"
    print(f"case 4 (step limit)     -> {result}")

# new in Step 3
def test_weather_uses_tool(model):
    task = "What's the weather in Lahore right now?"
    used = tools_used(run_worker(model, task, 5))
    result = "OK"
    if "get_weather" not in used:
        result = "WRONG"
    print(f"case 5 (weather task)   -> {result}: {used}")

# new in Step 3
def test_tool_failure_recovery(model):
    search_state.call_count = 0
    outcome = run_worker(model, "What is your refund policy?", 5)
    failed = 0
    succeeded = 0
    for step in outcome.steps:
        if step["error"] is None:
            succeeded = succeeded + 1
        else:
            failed = failed + 1
    result = "OK"
    if failed == 0 or succeeded == 0:
        result = "WRONG"
    print(f"case 6 (recovery)       -> {result}: {failed} failed, "
          f"{succeeded} succeeded")

if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    test_direct_answer(model)
    test_conversion_uses_tool(model)
    test_no_tool_needed(model)
```

```
    test_step_limit(model)
    # new in Step 3
    test_weather_uses_tool(model)
    test_tool_failure_recovery(model)
```

`main.py` doesn't change in this step — the new tools simply appear in the printed step log.

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_tools
python -m tests.unit.test_agents
python -m tests.integration.test_live_agents
python main.py
```

Expected output of the new `test_tools` cases:

```
case 3 (Lahore coordinates)       -> OK
case 4 (unknown city gives None)  -> OK
case 5 (fails once, then works)   -> OK
```

Expected output of `test_live_agents` — the Step 2 cases, then (one `WARNING` log line appears for the failed search):

```
case 5 (weather task)   -> OK: ['get_weather']
case 6 (recovery)       -> OK: 1 failed, 1 succeeded
```

Expected app session:

```
You: What is your refund policy?
... WARNING step 0: search_docs failed -> search index temporarily
unavailable
  step 0: search_docs({'query': 'refund policy'})
          -> Error: search index temporarily unavailable
  step 1: search_docs({'query': 'refund policy'})
          -> Refunds are accepted within 30 days of purchase, with the
             original receipt.
Answer: Refunds are accepted within 30 days of purchase, with the
original receipt.
```

(Long lines are wrapped here to fit the page.)